SUP'COM
Higher School of Communication of Tunis

Université de Carthage
جامعة قرطاج

SUMMER INTERNSHIP REPORT



Distributed Training on Kubeflow

carried out within

COGNIRA
COGNITIVE RETAIL ANALYTICS

BY
Camelia Ben Laamari

SUPERVISED BY
Salma Knani
Mona Tayahi

Sté COGNIRA TUNISIA
Imm. PREMIUM
Les Berges du Lac - 1053 Tunis
MF : 1459013 F / A / M / 000

ACADEMIC YEAR: 2020 - 2021

Table of contents

General Introduction 1

I. General Context 2

I.I. Company presentation 2

I.II. Project description 2

I.III. Internship tasks 2

II. Basic Concepts 4

II.I. kubernetes 4

II.I.I. Containers 5

II.I.II. Pods 5

II.II. Docker 5

II.II.I. Docker image 5

II.II.II. Docker container 6

II.III. Kubeflow 6

II.III.I. Kubeflow pipeline 6

II.III.III. Notebook servers 7

II.IV. volumes 7

II.IV.I. Persistent Volume 7

II.IV.II. Persistent Volume Claim 7

II.V. Tensorflow 8

III. Kubeflow Pipeline 9

III.I. Overview 9

III.II. Pipeline steps 10

III.II.I. Data preprocessing 10

III.II.II. Linear regression 11

III.II.III. DNN regression 11

III.II.IV. Evaluation 12

III.III. Pipeline run 12

IV. Distributed Training 17

IV.I. Types of distributed training 17

IV.I.I. Model parallelism 17

IV.I.II. Data parallelism 18
IV.II. Distributed training strategies 18
IV.II.I. Synchronous all-reduce strategies 18
IV.II.II. Asynchronous parameter server strategies 19
IV.III. TFJOB 21
IV.IV. implementation 21
IV.IV.I. Defining the strategy 21
IV.IV.II. Cluster configuration 22
V. Results Benchmarking 24
V.I. Performance evaluation 24
V.II. challenges and difficulties 25
Conclusion 27
Bibliography 28

List of figures

Figure 1. Cognira's Logo 2
Figure 2. Kubernetes cluster diagram 4
Figure 3. Kubeflow UI 6
Figure 4. Kubeflow notebook servers 7
Figure 5. The relationship between PV and PVC 8
Figure 6. Pipeline graph 10
Figure 7. Initial dataset 10
Figure 8. Preprocessed dataset 10
Figure 9. Training and validation losses of the linear model 11
Figure 10. DNN regression graph 11
Figure 11. Training and validation losses of the DNN model 12
Figure 12. Comparing MAE 12
Figure 13. Comparing RMSE 12
Figure 14. Uploading a pipeline 14
Figure 15. List of pipelines 14
Figure 16. Creating a new experiment 15
Figure 17. Creating a new pipeline run 15
Figure 18. Successful run of the pipeline 16
Figure 19. Types of distributed training 17
Figure 20. Sync all-reduce architecture 19
Figure 21. Parameter server architecture 20

Acknowledgement

First, I would like to express my thanks to SUP'COM and Cognira for providing me the opportunity to embark on this internship journey and expand my knowledge.

I sincerely thank my supervisors Ms. Mona Tayahi and Ms. Salma Knani for their guidance and encouragement in carrying out this project work. I also thank the rest of the R&D (Research & Development) Team for their collaboration and for their constructive feedback on my work.

Finally, I wish to express my gratitude to HR Manager, Ms. Soufia Ghariani, for the smooth onboarding, and to the other coworkers in Cognira, especially Ms. Rihab Ben Aicha, for rendering help during the period of my project work.

General Introduction

Retailers' sustainability depends, to a great extent, on their ability to forecast. Thanks to AI and automated machine learning techniques, retailers now have a better understanding of how a supply chain works, how to discover inefficiencies, and how to develop their business.

In fact, a machine learning model can analyze and break large volumes of complex data into actionable insights, so retailers can better understand customer behavior and market trends. Using these insights, they can easily estimate future demand, set competitive prices, personalize offerings for customers, and much more.

Indeed, big businesses and innovators like eBay, Amazon, and Alibaba have already embraced AI technologies in the sales cycle, from storage logistics to customer service. However, as models become more complicated and datasets grow larger, training them on a single machine becomes increasingly difficult. As a result, the idea of distributed training was developed, with the goal of sharing the workload across multiple machines to improve both performance and speed.

This document outlines the different tasks performed during this 7-week-internship within the R&D Team of Cognira Tunisia, about “Distributed Training on Kubeflow”. It starts by presenting the host company and the purpose of the project, then explains the concept of a machine learning pipeline and distributed training, and how to implement them using Kubeflow and Python.

I. General Context

I.I. COMPANY PRESENTATION

Cognira is an AI solutions provider for enterprise retailers, founded in 2015, in Atlanta, Georgia, by experienced data scientists and retail experts. It aims at helping retailers unlock valuable, transformative business insights from their data.



Figure 1. Cognira's Logo

Cognira was ranked 30th in The Financial Times' inaugural ranking of the fastest growing companies in the Americas in 2020 overall, and 9th among all technology companies.

It also made the Inc. 5000 list, the most prestigious ranking of the USA's fastest-growing private companies, for 2 years consecutively (2019 and 2020).

I.II. PROJECT DESCRIPTION

This internship is the continuation of a previous end-of-studies project about automating a machine learning lifecycle. This process is also known as MLOps or ML Operations and it tends to apply the DevOps principles to productionalize machine learning models through practices such as testing, continuous delivery and monitoring. MLOps aims to help improve the consistency as well as the quality of the machine learning solutions.

I.III. INTERNSHIP TASKS

The internship program is as follows:

1. Getting started on Tensorflow:
    a. Preparing a simple code that trains models
2. Getting started with Kubeflow:
    a. Running existing code on Kubeflow (learn how to build and run some steps)
    b. Running the training code on Kubeflow
3. Distributed training:
    a. Researching strategies to distribute the training
    b. Updating the training code from step (1a) to implement the strategies
    c. Running the distribution through the UI in Kubeflow
    d. Benchmarking of results

II. Basic Concepts

II.I. KUBERNETES

Kubernetes, also known as K8s, is an open-source system for automating deployment, scaling, and management of containerized applications.
Kubernetes coordinates a highly available cluster of computers that are connected to work as a single unit. The abstractions in Kubernetes allow the deployment of containerized applications to a cluster without tying them specifically to individual machines.

To make use of this new model of deployment, applications need to be packaged in a way that decouples them from individual hosts: they need to be containerized. Containerized applications are more flexible and available than in past deployment models, where applications were installed directly onto specific machines as packages deeply integrated into the host.

A Kubernetes cluster consists of two types of resources:

- The Control Plane that coordinates the cluster
- Nodes (workers) that run applications

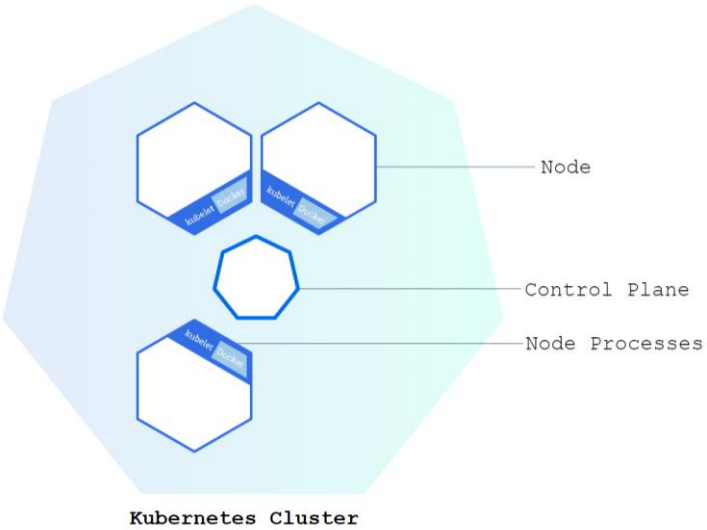


Figure 2. Kubernetes cluster diagram

II.I.I. Containers

Containers are a type of virtualized operating systems. A single container can run everything from a small microservice to a huge application. All necessary executables, binary code, libraries, and configuration files are encapsulated within a container.
Containers, unlike servers or virtual machines, do not include operating system images. As a result, they are lighter and more portable, with much less overhead. Multiple containers can be deployed in a Kubernetes cluster.

II.I.II. Pods

A Pod is a group of one or more containers that share storage and network resources and follow a set of rules for how to run them. The contents of a Pod are always co-located, co-scheduled, and executed in the same environment. A Pod represents an application-specific "logical host": it contains one or more tightly connected application containers.

II.II. DOCKER

Docker is a set of platform-as-a-service (PaaS) products that use OS-level virtualization to deliver software in packages called containers. Containers are isolated from one another and bundle their own software, libraries and configuration files. Because all of the containers share the services of a single operating system kernel, they use fewer resources than virtual machines.

II.II.I. Docker image

An image consists of a collection of files (or layers) that pack together all the necessities—such as dependencies, source code, and libraries—needed to set up a completely functional container environment. Images are stored on a Docker registry, such as the Docker Hub, or on a local registry.

II.II.II. Docker container

A Docker container is a virtualized runtime environment that provides isolation capabilities for separating the execution of applications from the underpinning system. It's an instance of a Docker image.

Containers provide a portable and lightweight environment for deploying applications. Each container is autonomous and runs in its own isolated environment, ensuring it does not disrupt other running applications or its underlying system. This greatly improves the security of applications.

II.III. KUBEFLOW

Kubeflow is a free and open-source project, designed to make running Machine Learning workflows on Kubernetes clusters simpler and more coordinated. This is a Cloud-Native framework for employing Machine Learning in containerized environments in Kubernetes.

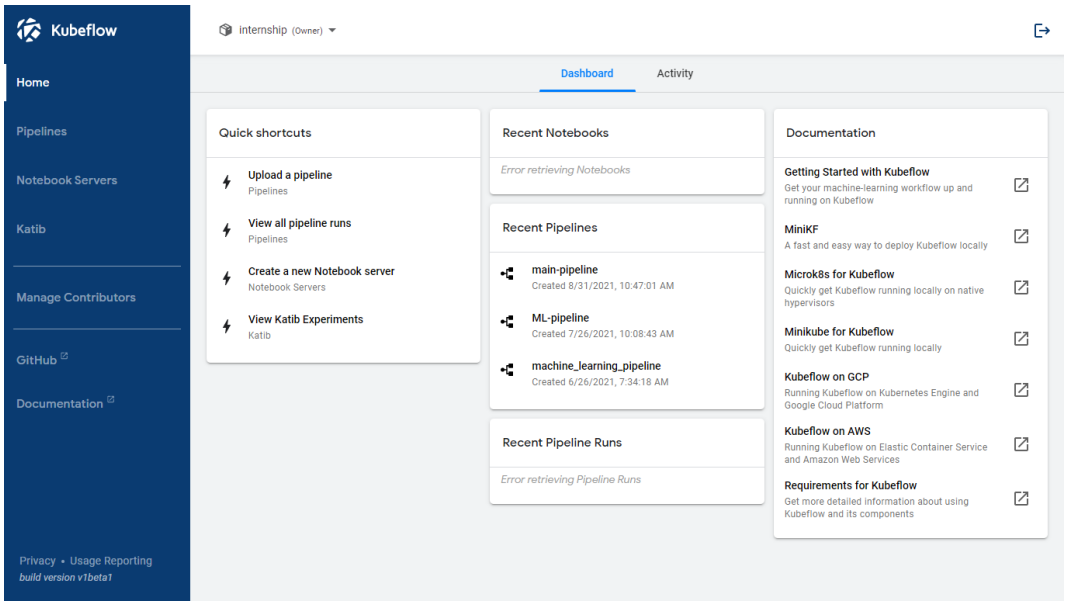


Figure 3. Kubeflow UI

II.III.I. Kubeflow pipeline

Kubeflow Pipelines is a platform for building and deploying portable, scalable machine learning (ML) workflows based on Docker containers.

A pipeline component is a self-contained set of user code, packaged as a Docker image, that performs one step in the pipeline.

II.III.III. Notebook servers

Kubeflow enables users to create notebook containers or pods directly in the cluster, rather than locally on their workstations, by integrating Jupyter notebooks. Admins can provide standard notebook images for their organization, and set up Role-Based Access Control (RBAC), secrets and credentials to manage which teams and individuals can access the notebooks.

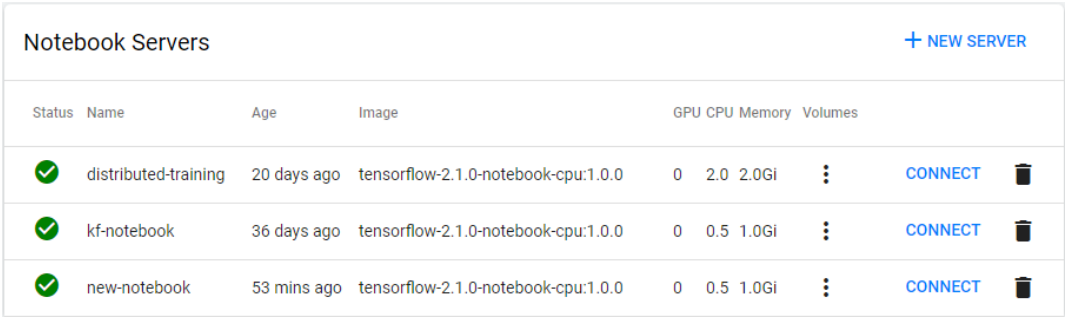


Figure 4. Kubeflow notebook servers

II.IV. VOLUMES

II.IV.I. Persistent Volume

A PersistentVolume (PV) is a piece of storage in a cluster that has been provided either manually or dynamically using Storage Classes. It functions as a cluster resource in the same way that a node does. PVs are volume plugins similar to Volumes, but their lifecycle is independent of the Pods that use them.

II.IV.II. Persistent Volume Claim

A user's request for storage is referred to as a PersistentVolumeClaim (PVC). It is similar to a Pod. Node resources are consumed by pods, while PV resources are

consumed by PVCs. Specific resource levels can be requested by pods (CPU and Memory). Specific size and access modalities are available in claims (e.g., they can be mounted ReadWriteOnce, ReadOnlyMany or ReadWriteMany, see AccessModes).

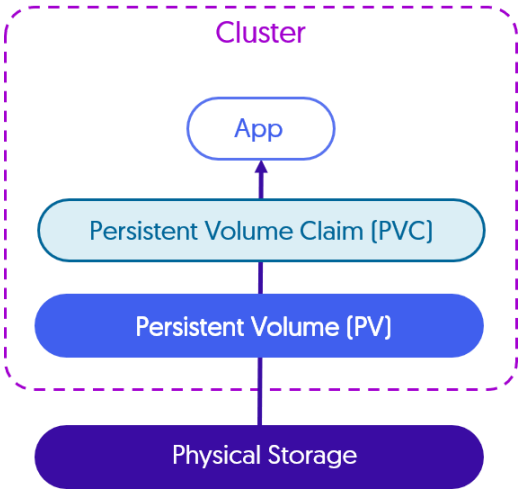


Figure 5. The relationship between PV and PVC

II.V. TENSORFLOW

TensorFlow is an end-to-end open source platform for machine learning. It has a comprehensive, flexible ecosystem of tools, libraries and community resources that lets researchers push the state-of-the-art in ML and developers easily build and deploy ML powered applications.

It enables users to build and train ML models easily using intuitive high-level APIs like Keras with eager execution, which makes for immediate model iteration and easy debugging.

III. Kubeflow Pipeline

III.I. OVERVIEW

A machine learning pipeline is a way to codify and automate the workflow it takes to produce a machine learning model. ML pipelines consist of multiple sequential steps that do everything from data extraction and preprocessing to model training and deployment.

The model we're training in this project performs a time-series forecasting of the sales units in the future, given around 2 years of sales data and some useful attributes. The provided CSV dataset is from a previous Cognira contest and contains multiple attributes (features) like the week, the store ID, the district, etc, from which we have to predict the number of units (label).

Detail Compact Column 20 of 20 columns

# ITEM	# STORE	id	# UNITS	# DISTRICT	# AREA	# CHAIN
2202490	10832	1	0.0	42481212	1616	16
2202490	10832	1	0.0	42481212	1616	16
2202490	10832	1	0.0	42481212	1616	16
2202490	10832	1	0.0	42481212	1616	16
2202490	10832	1	0.0	42481212	1616	16
2202490	10832	1	0.0	42481212	1616	16

The steps for the model training pipeline are:

- Data preprocessing
- Linear regression
- DNN regression
- Models' evaluation

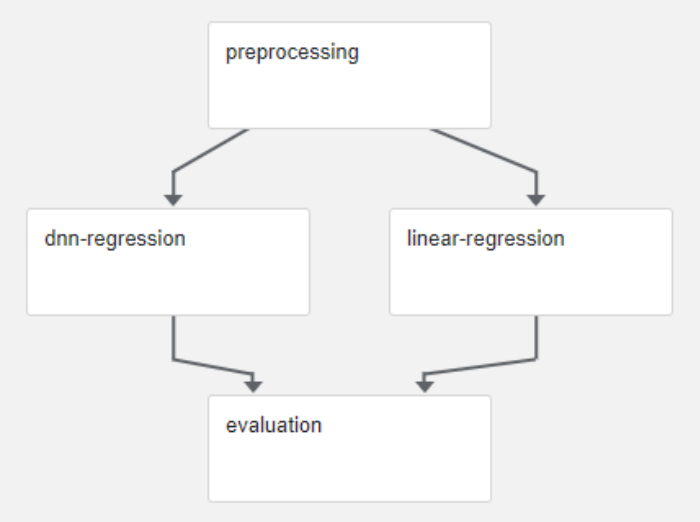


Figure 6. Pipeline graph

III.II. PIPELINE STEPS

III.II.I. Data preprocessing

In the preprocessing step, the data is cleaned and converted to numeric-only, by removing irrelevant columns (duplicate columns and index), converting boolean data to binary (1 for `True` and `0` for `False`) and replacing NaN values by 0.

	Unnamed: 0	WEEK	ITEM	STORE	id	UNITS	DISTRICT	AREA	CHAIN	SUBCLASS	CLASS	DEPT	GROUP_NO	COMPANY	OOS_IND	VERY HIGH
0	0	W36_2016	2202490	10832	1	0.0	42481212	1616	16	30372882	4049716	505	600	111	false	false
1	1	W36_2016	2202490	10832	1	0.0	42481212	1616	16	30372882	4049716	505	600	111	false	false
2	2	W36_2016	2202490	10832	1	0.0	42481212	1616	16	30372882	4049716	505	600	111	false	false
3	3	W36_2016	2202490	10832	1	0.0	42481212	1616	16	30372882	4049716	505	600	111	false	false
4	4	W36_2016	2202490	10832	1	0.0	42481212	1616	16	30372882	4049716	505	600	111	false	false

Figure 7. Initial dataset

	ITEM	STORE	id	UNITS	DISTRICT	AREA	CHAIN	SUBCLASS	CLASS	DEPT	GROUP_NO	COMPANY	OOS_IND	VERY HIGH	HIGH
0	-0.90636	2.636179	-1.728278	-0.177265	-0.409748	1.313955	-0.957672	0.426557	0.439322	5.05	6.0	1.11	-0.370322	0.0	-0.058981
1	-0.90636	2.636179	-1.728278	-0.177265	-0.409748	1.313955	-0.957672	0.426557	0.439322	5.05	6.0	1.11	-0.370322	0.0	-0.058981
2	-0.90636	2.636179	-1.728278	-0.177265	-0.409748	1.313955	-0.957672	0.426557	0.439322	5.05	6.0	1.11	-0.370322	0.0	-0.058981
3	-0.90636	2.636179	-1.728278	-0.177265	-0.409748	1.313955	-0.957672	0.426557	0.439322	5.05	6.0	1.11	-0.370322	0.0	-0.058981
4	-0.90636	2.636179	-1.728278	-0.177265	-0.409748	1.313955	-0.957672	0.426557	0.439322	5.05	6.0	1.11	-0.370322	0.0	-0.058981

Figure 8. Preprocessed dataset

III.II.II. Linear regression

In this step we use a multiple linear regression to model the relationship between the features and the label by fitting a linear equation to the observed data.

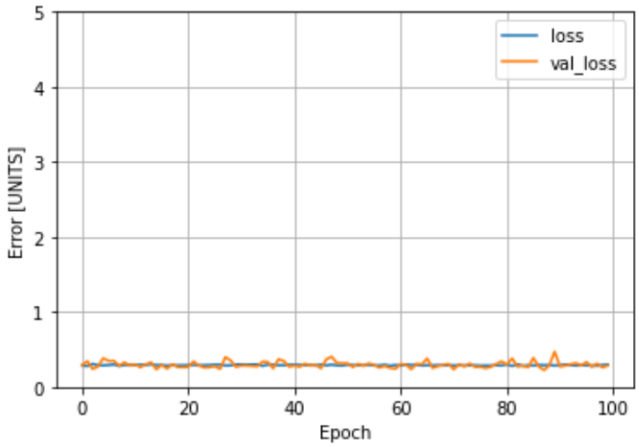


Figure 9. Training and validation losses of the linear model

III.II.III. DNN regression

The DNN regression model is composed of 2 hidden layers of 60 nodes each, using ReLU activation function, and the output layer.

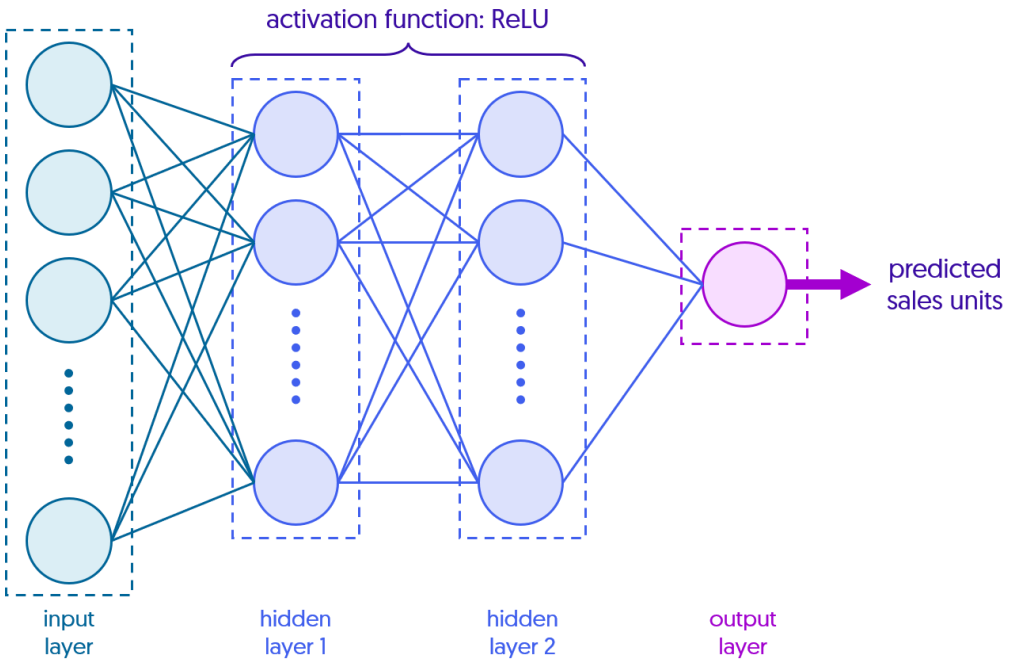


Figure 10. DNN regression graph

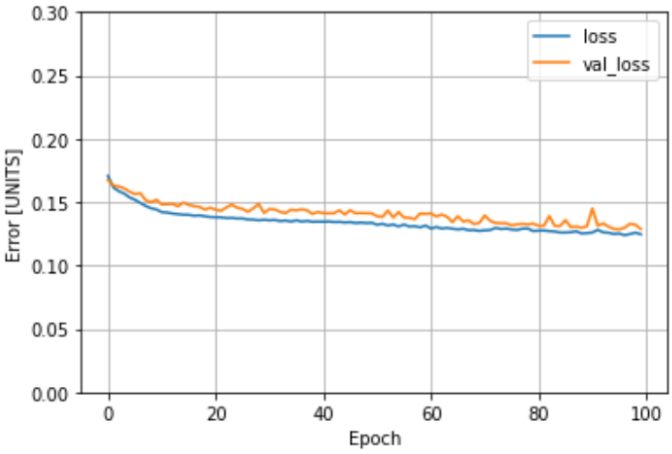


Figure 11. Training and validation losses of the DNN model

III.II.IV. Evaluation

The final pipeline step is for evaluating the linear and DNN models' performance using RMSE (Root Mean Square Error) and MAE (Mean Absolute Error).

	Mean absolute error [UNITS]
linear_model	0.290036
dnn_model	0.130230

Figure 12. Comparing MAE

	Root Mean Square Error [UNITS]
linear_model	0.157311
dnn_model	0.077396

Figure 13. Comparing RMSE

III.III. PIPELINE RUN

To build the Kubeflow pipeline, we need to containerize the components. We build a docker image for each pipeline step from a `Dockerfile`, and specify the base image that contains the necessary Python modules to run the scripts.

Below is the `Dockerfile` of the preprocessing step:

```
FROM devcognira.azurecr.io/internship/base-python-packages:latest
COPY dataset.csv ./
COPY preprocess.py ./
CMD  ["python", "preprocess.py"]
```

We then proceed by building the image using the command:

```
docker build -t devcognira.azurecr.io/internship/preprocessing:latest .
```

and push it to the container registry:

```
docker push devcognira.azurecr.io/internship/preprocessing:latest
```

In the main pipeline file, we use `ContainerOp` method to convert the Python script into a pipeline component, and specify the docker image and arguments.

```
# Preprocessing component
    operations['preprocessing'] = dsl.ContainerOp(
        name = 'preprocessing',
        image = 'devcognira.azurecr.io/internship/preprocessing:latest',
        command = ['python'],
        arguments = [
            './preprocess.py',
            '--data_path', data_path,
            '--persistent_volume_path', persistent_volume_path
        ]
    )
```

In order to be able to share data between different components, we have to create a persistent volume to store the output of each pipeline step, so that the next one can access it.

Finally, we need to schedule the components as follows:

```
operations['linear_regression'].after(operations['preprocessing'])
```

before compiling the pipeline into a `.tar.gz` file (that we'll be using later):

```
compiler.Compiler().compile(main_pipeline, __file__ + '.tar.gz')
```

Once the compressed pipeline file is generated, we navigate to the Kubeflow UI and upload a new pipeline:

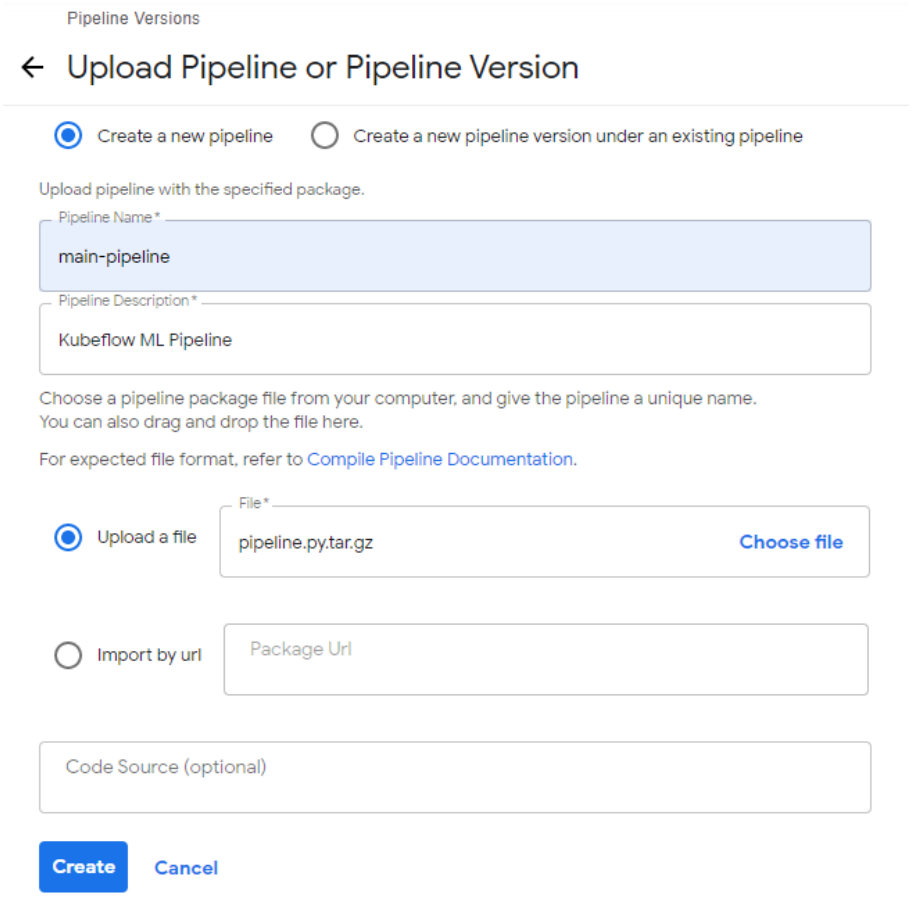


Figure 14. Uploading a pipeline

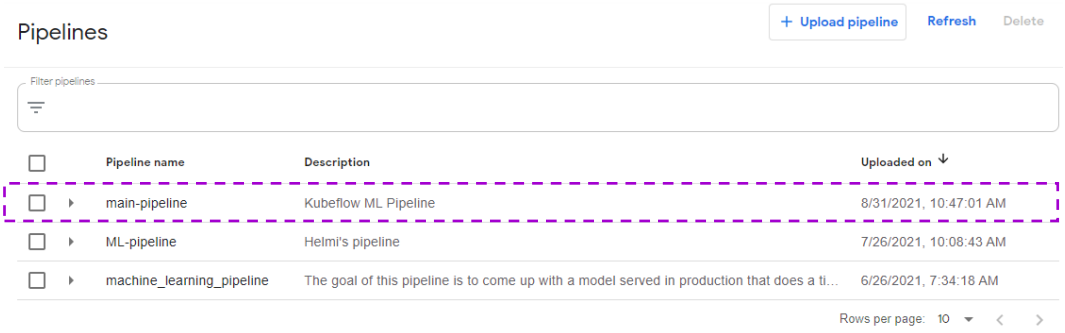


Figure 15. List of pipelines

Next, we create a new experiment. An experiment is a workspace where users can try different pipeline configurations:

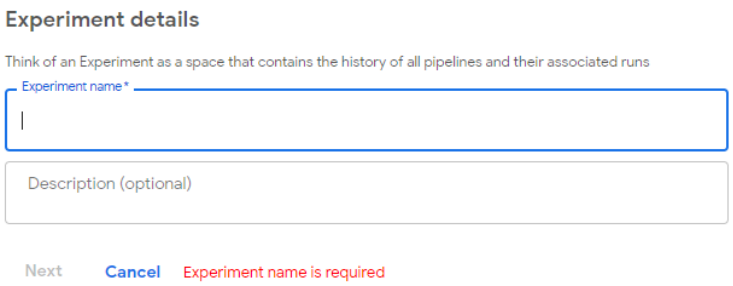


Figure 16. Creating a new experiment

We create a new pipeline run inside the new experiment, and specify the input parameters of the pipeline:

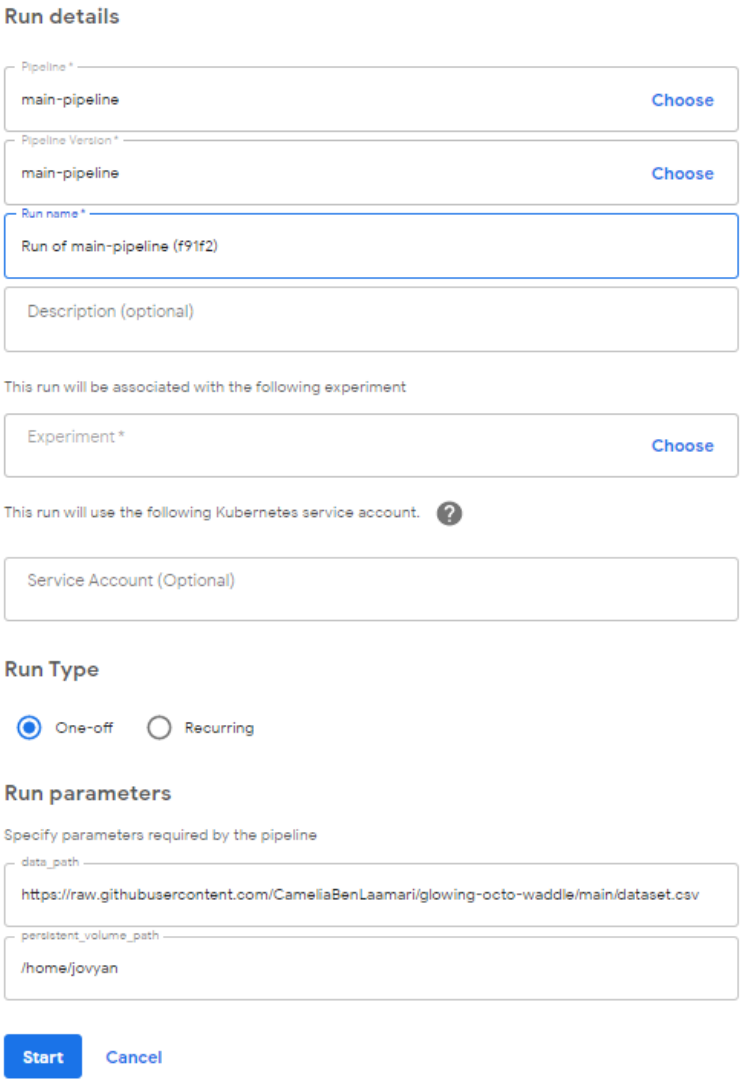


Figure 17. Creating a new pipeline run

As soon as we start the run, the pods where the pipeline components are containerized start initializing. Once they complete running, we can see the status (successful or not) of each pipeline step:

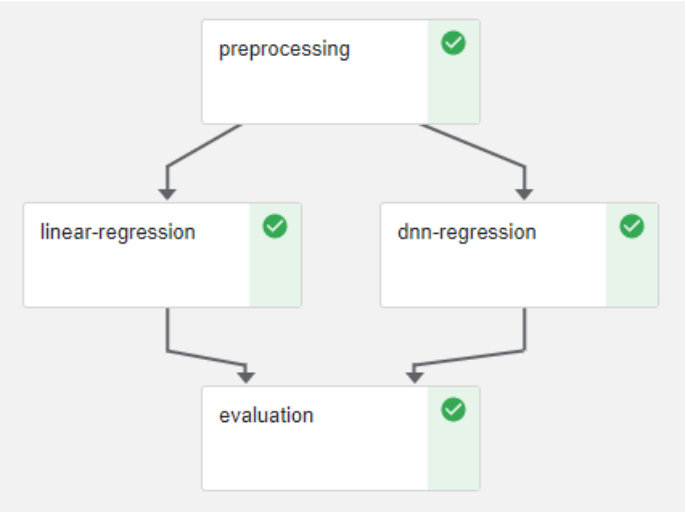


Figure 18. Successful run of the pipeline

IV. Distributed Training

Training a machine learning model can take a long time. As dataset sizes grow larger, it becomes increasingly difficult to train models within a limited time frame. Distributed training approaches are used to overcome this kind of challenge. The training workloads are split up across multiple mini-processors. These mini-processors, or worker nodes, work in parallel to speed up the training process, making it possible to train very large models.

IV.I. TYPES OF DISTRIBUTED TRAINING

There are 2 different ways to distribute the training:

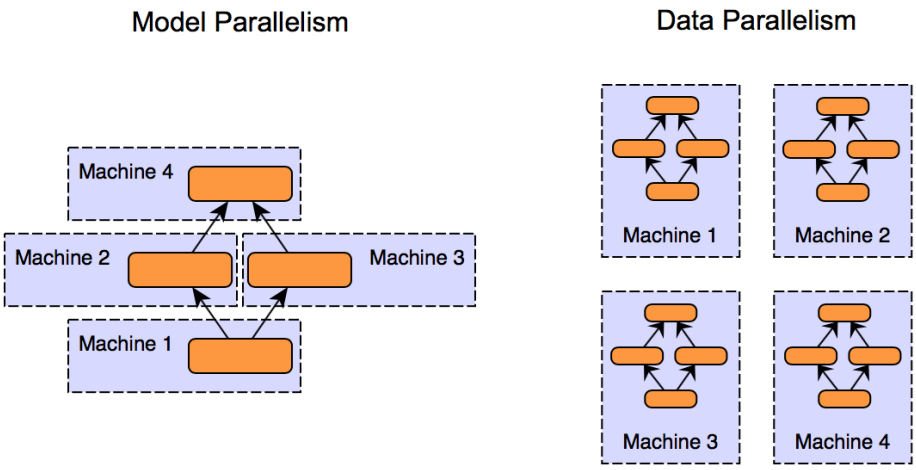


Figure 19. Types of distributed training

IV.I.I. Model parallelism

Model parallelism divides the model into sections that are trained simultaneously on multiple worker nodes. Workers use the same data set and only need to communicate global model parameters with other workers when forward or backward propagation is being performed. Model parallelism is substantially more complex to build, and only works in models with naturally parallel architectures, such as those with many branches.

IV.I.II. Data parallelism

In data parallelism, the data is split up and analyzed in parallel. Each worker node trains a copy of the model on a different batch of training data, then communicates the results after computation to maintain all nodes' model parameters and gradients in sync. These results can be communicated synchronously (at the end of each batch computation) or asynchronously (at any time during the batch computation).

In TensorFlow, distributed training is built around data parallelism. `tf.distribute.Strategy` is a TensorFlow API to distribute training across multiple machines, GPUs or TPUs.

IV.II. DISTRIBUTED TRAINING STRATEGIES

IV.II.I. Synchronous all-reduce strategies

Each worker starts off with its own identical copy of the full model and performs each training step on a different subset of the training data. It then broadcasts its resultant gradients to all of the other workers, and updates its model weights based on the gradients calculated by all of the workers.

There are three common sync training strategies:

Mirrored Strategy	One machine having multiple GPUs	Each model is trained on different slices of data and weight updating is done using efficient cross-device communication algorithms (all-reduce algorithms) such as Hierarchical copy, reduction to one device, NCCL (default).

TPU Strategy	One machine having multiple TPUs	Same as Mirrored Strategy
Multi-Worker Mirrored Strategy	Multiple machines having multiple GPUs	Workers are run in lock-step and synchronizing the gradient at each step. Either ring all-reduce or hierarchical all-reduce communication used for cross-device communication.

The main problem with synchronous distributed training, is that each worker has to wait for all the other workers to finish computing and aggregating the parameters, before training on the next batch of data, which can slow down the training process.

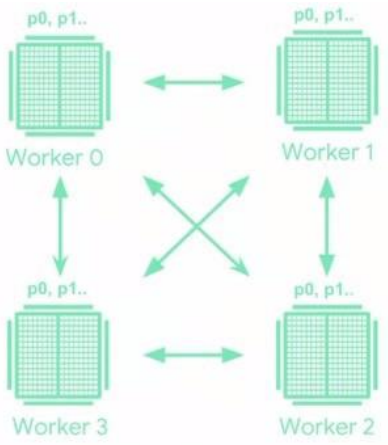


Figure 20. Sync all-reduce architecture

IV.II.II. Asynchronous parameter server strategies

All workers train independently over the input data and update variables asynchronously. There are two common asynchronous strategies:

Parameter Server Strategy	Multiple machines	In this setup, some machines are designated as workers and some as parameter servers Each variable of the model is placed on one parameter server Computation is replicated across all GPUs of all the workers. Worker tasks are read input, variables, compute forward and backward and send updates to the parameter servers.
Central Storage Strategy	One machine having multiple GPUs	Variables are not mirrored, instead they are placed on the CPU and operations are replicated on all local GPUs

The main disadvantage of asynchronous distributed training is the risk of divergence. In fact, as soon as a worker finishes training on a batch of dataset, it automatically starts training on the next one. Unless all the other workers aggregate the gradients simultaneously, the worker risks training with old parameters, hence the divergence in the training process.

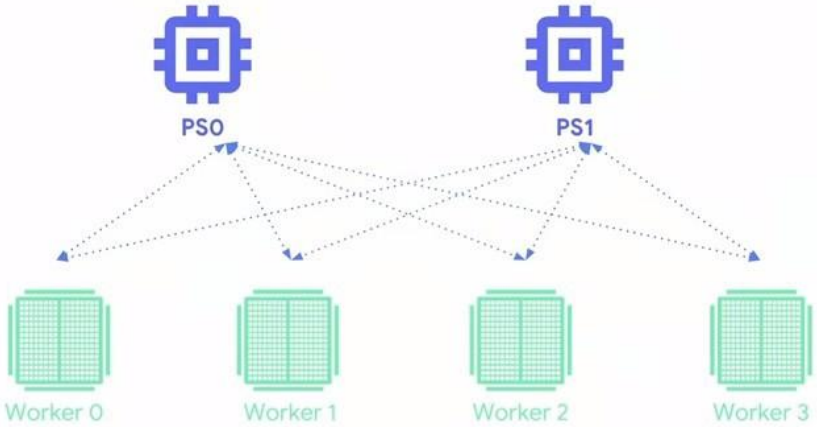


Figure 21. Parameter server architecture

IV.III. TFJOB

TFJob is a Kubernetes custom resource to run TensorFlow training jobs on Kubernetes. It is a resource with a YAML representation like the following:

```yaml
apiVersion: kubeflow.org/v1
kind: TFJob
metadata:
  generateName: tfjob
  namespace: your-user-namespace
spec:
  tfReplicaSpecs:
    Worker:
      replicas: 3
      restartPolicy: OnFailure
      template:
        metadata:
          annotations:
            sidecar.istio.io/inject: "false"
        spec:
          containers:
            - name: tensorflow
              image: gcr.io/your-project/your-image
              command:
                - python
                - -m
                - trainer.task
                - --batch_size=32
                - --training_steps=1000
```

In the example below, the TFJob is scheduled to be distributed among 3 workers.

IV.IV. IMPLEMENTATION

IV.IV.I. Defining the strategy

To use a distributed training strategy, we have to use `tf.distribute` API:

```
strategy = tf.distribute.MirroredStrategy()
```

Next, to split up the dataset on the workers, we need to shuffle it and batch it, so that each worker trains on a batch:

```
train_dataset = train_dataset.shuffle(BUFFER_SIZE).batch(BATCH_SIZE)
```

Some strategies like the Parameter Server Strategy requires a cluster configuration.

IV.IV.II. Cluster configuration

The `TF_CONFIG` environment variable is the standard way in TensorFlow to specify the cluster configuration to each worker that is part of the cluster. It is basically a JSON string which specifies what tasks constitute a cluster, their addresses and each task's role in the cluster.

Valid roles in the cluster are:

- "`Chief`": that orchestrates the training and performs tasks like checkpointing the model.
- "PS" or Parameter Server: that provide a distributed data store for the model parameters.
- "`Worker`": that does the actual training job. By default, `Worker0` acts as the chief.
- "`Evaluator`": that compute evaluation metrics as the model is trained.

The cluster spec returned will be initialized from the `TF_CONFIG` environment variable. However, sometimes the container orchestration framework sets `TF_CONFIG` for the user. That's why we need to reset it if we want to create our own cluster configuration, using:

```
os.environ.pop('TF_CONFIG', None)
```

Then, we need to specify the workers and PS, and assign them to ports using portpicker module:

```
worker_ports = [portpicker.pick_unused_port() for _ in range(num_workers)]
ps_ports = [portpicker.pick_unused_port() for _ in range(num_ps)]

cluster_dict = {}
cluster_dict["worker"] = ["localhost:%d" % _ for _ in worker_ports]
cluster_dict["ps"] = ["localhost:%d" % _ for _ in ps_ports]


cluster_spec = tf.train.ClusterSpec(cluster_dict)
```

Finally, we create the cluster resolver from the previous cluster spec:

```
cluster_resolver = tf.distribute.cluster_resolver.SimpleClusterResolver(
    cluster_spec,
    rpc_layer="grpc"
)
```

Once the strategy defined and the `TFJobs` created, we apply them using `kubectl`, the command line tool of Kubernetes:

```
kubectl apply -f <tfjob-yaml-file-path>
```

V. Results Benchmarking

V.I. PERFORMANCE EVALUATION

After experimenting with the different strategies, it is clear how distributed training accelerates the training process. Indeed, the example below shows how increasing the number of PS and workers in the Parameter Server Strategy minimizes both the loss and the mean squared error of the linear regression.

Linear regression with 1 PS and 2 workers:

```
Epoch 1/12
100/100 - 5s - loss: 4.3827 - mean_squared_error: 20.5297
Epoch 2/12
100/100 - 1s - loss: 2.9784 - mean_squared_error: 10.4941
Epoch 3/12
100/100 - 1s - loss: 1.7013 - mean_squared_error: 4.4094
Epoch 4/12
100/100 - 1s - loss: 0.9176 - mean_squared_error: 1.7694
Epoch 5/12
100/100 - 2s - loss: 0.5076 - mean_squared_error: 1.3849
Epoch 6/12
100/100 - 1s - loss: 0.2906 - mean_squared_error: 1.3268
Epoch 7/12
100/100 - 1s - loss: 0.1803 - mean_squared_error: 0.9952
Epoch 8/12
100/100 - 2s - loss: 0.1392 - mean_squared_error: 0.9257
Epoch 9/12
100/100 - 2s - loss: 0.1149 - mean_squared_error: 0.6290
Epoch 10/12
100/100 - 2s - loss: 0.1257 - mean_squared_error: 2.9614
Epoch 11/12
100/100 - 2s - loss: 0.1030 - mean_squared_error: 0.8479
Epoch 12/12
100/100 - 1s - loss: 0.0866 - mean_squared_error: 0.6557
```

Linear regression with 2 PS and 4 workers:

```
Epoch 1/12
100/100 - 8s - loss: 0.7912 - mean_squared_error: 1.9465
```

```
Epoch 2/12
100/100 - 2s - loss: 0.5425 - mean_squared_error: 1.5099
Epoch 3/12
100/100 - 2s - loss: 0.3527 - mean_squared_error: 0.8780
Epoch 4/12
100/100 - 3s - loss: 0.2261 - mean_squared_error: 1.0126
Epoch 5/12
100/100 - 3s - loss: 0.1321 - mean_squared_error: 0.8028
Epoch 6/12
100/100 - 3s - loss: 0.1200 - mean_squared_error: 0.9708
Epoch 7/12
100/100 - 2s - loss: 0.1065 - mean_squared_error: 0.7698
Epoch 8/12
100/100 - 4s - loss: 0.0831 - mean_squared_error: 0.5077
Epoch 9/12
100/100 - 3s - loss: 0.0927 - mean_squared_error: 0.8048
Epoch 10/12
100/100 - 5s - loss: 0.0935 - mean_squared_error: 0.9012
Epoch 11/12
100/100 - 5s - loss: 0.0889 - mean_squared_error: 0.7333
Epoch 12/12
100/100 - 3s - loss: 0.1245 - mean_squared_error: 1.7637
```

However, we can notice the slight increase in the training time per epoch. That can be explained by the RPC (Remote Procedure Call) protocol used to communicate between the PS and the workers, that slows down the training process.

V.II. CHALLENGES AND DIFFICULTIES

One of the main difficulties I have faced during this internship, is the lack of documentation resources. The official Kubeflow and Tensorflow documentation are very basic and barely cover any of the main issues, not to mention that many articles are outdated and haven't been updated to match the latest software releases.
To overcome this problem, I have resorted to Stack Overflow and Github Community where I could make the most out of experts' advice.

Moreover, since the Kubernetes volumes are of type HostPath, they can only be accessed through ReadWriteOnce mode, which requires a single-node cluster. At

first, this caused a problem when running distributed training strategies, since the pods get scheduled on different nodes.

As a workaround, I added a label to one of the nodes, and used `node affinity` to schedule all the pods on the same node. Even though it's not the best practice since a big number of workers can cause the node's saturation and prevent the pods from running, it was a temporary hack that helped me carry on my experiments.

Conclusion

This project, entitled “Distributed Training on Kubeflow”, is a deep dive into the MLOps field. Using Kubeflow and Tensorflow’s distributed training API, `tf.distribute`, the goal is to find a way to train complex models with massive datasets, in order to assist retailers make the most of their data to forecast future demand and grow their business.

This document highlights the importance of breaking the model training code into smaller containerized pipeline steps to automate the workflow, and gives an insight into how distributed training accelerates the training process and reduces the development cycle time.

For future improvement, I believe it would be better to launch the `TFJobs` inside the Python script—instead of manually applying them—making it possible to automate the distributed training pipeline and run it through the Kubeflow UI.

This internship project was a great opportunity to master tools like Kubeflow, Docker and Tensorflow, and to apply my theoretical knowledge. It also helped me develop my observation, analysis and programming skills. Last but not least, this internship was a good introduction to the MLOps field and to the working environment overall.

Bibliography

"Machine Learning In Retail: 5 Examples Of Successful Implementation". *Massmediagroup.Pro*, 2021, https://massmediagroup.pro/blog-mmg/5-examples-of-machine-learning-in-retail#:~:text=With%20AI%20and%20automated%20machine,purchases%2C%20inventory%2C%20and%20sales. Accessed 9 Sept 2021.

"Tensorflow Training (Tfjob)". *Kubeflow*, 2021, https://www.kubeflow.org/docs/components/training/tftraining/. Accessed 8 Sept 2021.

"Production-Grade Container Orchestration". *Kubernetes*, 2021, https://kubernetes.io/. Accessed 10 Sept 2021.

"Distributed Training - Run:AI". *Run:AI*, 2021, https://www.run.ai/guides/gpu-deep-learning/distributed-training/. Accessed 10 Sept 2021.

"Distributed Training With Tensorflow.". *Medium*, 2021, https://medium.com/@deepakec1031/distributed-training-with-tensorflow-1e586d7fbcd6. Accessed 9 Sept 2021.